

AgentEmulator (v1.0) 使用说明

1. AgentEmulator 简介

1.1 AgentEmulator 是什么？

AgentEmulator 是由中山大学·软件工程学院·黄华威研究组（[HuangLab](#)）发起的、面向 AI 智能体可信基础设施的仿真与实验平台。平台以区块链作为可信记录与结算的基础，旨在帮助研究者和学生围绕智能体的身份、行为审计、支付结算、激励与治理机制开展实验，逐步形成支持 AI 智能体可信交互与协作的研究工具。

AgentEmulator 是面向 AI 智能体行为与区块链相结合的场景、基于 BlockEmulator-X 构建的**实验工具**。其中，BlockEmulator-X 是 HuangLab 于2026年6月开源的区块链仿真实验工具，是初代 BlockEmulator 的升级迭代版本，其 GitHub 代码仓库地址为 github.com/HuangLab-SYSU/block-emulator-x。

1.2 AgentEmulator 有什么用？

AgentEmulator 实验平台的**设计目标**是简化 AI Agent 相关的实验环境搭建配置、机制验证和数据分析，使实验人员能够轻松地配置底层区块链环境、观察 AI 智能体的行为，并通过实验数据分析在不同机制控制时的运行效果。

下图展示了 AgentEmulator 用户视角的工作流程图。

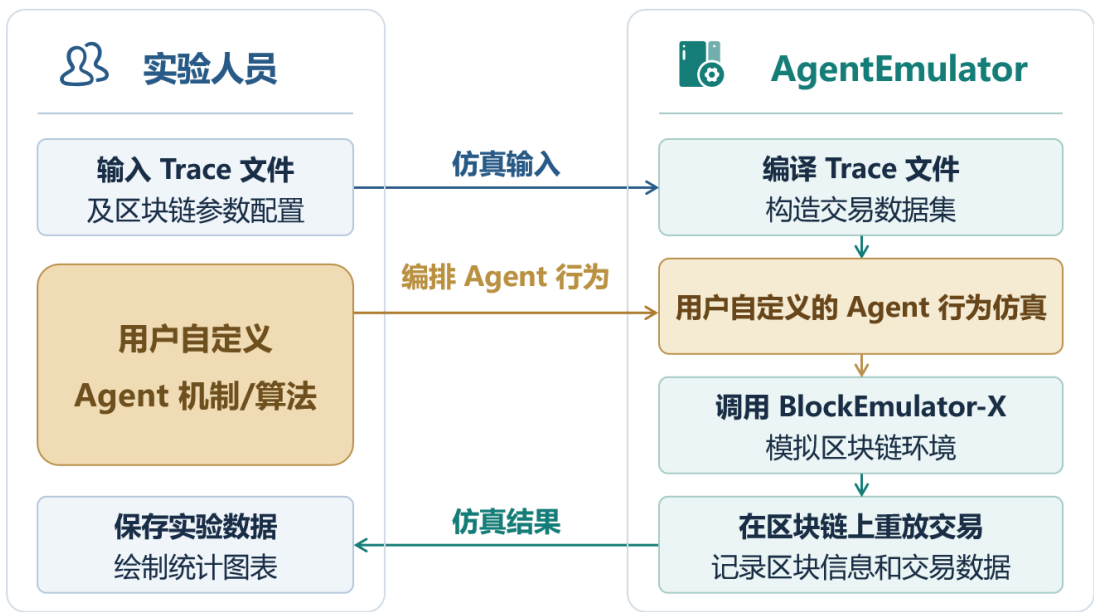


图 1. AgentEmulator 的 general purpose (并不只是对应于当前 v1.0 版本) 展示了用户与 AgentEmulator 之间的交互关系。其中，“用户自定义 机制/算法”具有非常大的自由发挥空间，是用户二次开发、自由创新之地。

1.3 发展路线 / Roadmap

AgentEmulator 将围绕 AI Agent 的“身份”“结算”“审计”“激励”“治理”五个方向逐步扩展。以下为拟推进的发展完善路线，具体安排将随相关研究与开发进展进行微调。

阶段	主要内容
v1.0：基础行为仿真（当前版本）	支持 Trace 文档驱动的“加入 (Join)”、“支付 (pay)”和“退出 (leave)”行为回放，提供交易执行、行为与交易映射、账户数据记录及结果可视化。
近期：完善身份与审计能力	逐步实现 DID 注册与注销的智能合约状态管理，扩展权限声明、可验证行为日志和行为追溯能力。
中期：扩展支付与交互机制	引入微支付、支付通道和批量结算等机制，扩展多轮反馈实验，支持更丰富的 Agent 服务交互场景。
后续：支持激励与协作治理实验	研究多 Agent 协作中的任务分配、行为协调与责任追溯机制，支持不同协作策略的仿真与效果评估。
长期：建设基准评测平台	积累标准化 Agent 相关的实验场景、数据集与评测指标，支持不同可信基础设施方案的比较和可复现实验。

1.4 当前发布版本 v1.0

当前发布的是 AgentEmulator v1.0，支持基于 Trace（实验输入数据）的基础行为回放、身份注册与注销留痕、逐笔直接支付，以及实验数据记录和可视化。实验人员使用 JSONL 格式的 Trace 文件描述 Agent 的“加入”、“转账”和“退出”行为；AgentEmulator 中的 agentSupervisor 模块将 Trace 文件编译为交易数据集，并调用 BlockEmulator-X 启动区块链环境、执行交易、并将交易执行结果上链记录。实验完成后，AgentEmulator 系统会记录交易与 Agent 账户的相关数据，自动绘制余额变化图集，并生成支持中英文切换的 HTML 实验结果页面（由启动脚本在默认浏览器中打开展示）。

AgentEmulator v1.0 中，Agent 行为由 Trace 文件预先定义，采用“用户原始指定顺序”（User-specified Original Sequence）策略，按照实验输入中指定的交易顺序执行交易，尚未集成额外的交易编排机制或调度算法。实验人员可在此基础上探索和实现自定义机制，例如调整交易的执行顺序，或根据交易优先级、Agent 权重等规则编排交易。

AgentEmulator 后续版本将持续升级迭代，逐步扩展协议、实验场景和评测能力。欢迎实验人员结合具体研究需求扩展示例代码，设计并验证不同机制或算法对实验结果的影响。

1.5 GitHub 代码仓库地址

GitHub 代码仓库地址为：<https://github.com/HuangLab-SYSU/agent-emulator>

1.6 术语解释

本节介绍 AgentEmulator 中的主要术语，帮助实验人员理解系统组成、输入文件和配置文件之间的关系。AgentEmulator 的基本工作流程是：**实验人员通过 Trace 文件描述 Agent 行为；AgentEmulator 中的 agentSupervisor 模块将 Trace 文件编译为交易数据集，并调用 BlockEmulator-X 执行其中的交易；AgentEmulator 随后记录实验数据并展示实验结果。**

术语	说明
AgentEmulator	基于 BlockEmulator-X 构建的 Agent 行为仿真模拟器，将输入的 Agent 行为转换为区块链交易并记录上链，自动记录实验日志并展示实验结果。
Agent	实验中执行行为的主体，可按照 Trace 文件中的描述执行“加入”、“支付”和“退出”操作。
BlockEmulator-X	AgentEmulator 的底层区块链仿真平台，负责节点运行、共识出块、交易执行和链上数据记录。（GitHub 代码仓库地址为 github.com/HuangLab-SYSU/block-emulator-x ）
Trace 文件	描述 Agent 行为序列的实验输入文件，采用 JSONL 格式，每行记录一次 Agent 行为或一笔普通转账。
agentEmuConfig.yaml 文件	AgentEmulator 的实验配置文件，用于指定 Trace 文件、身份派生seed、结果目录，以及是否启动区块链等参数。
config.yaml 文件	BlockEmulator-X 的区块链配置模板，用于设置区块链的分片数量、节点数量、出块间隔等底层参数。
agentSupervisor	AgentEmulator 的实验调度模块，负责读取配置和 Trace 文件、将行为编译为交易数据集，并组织底层区块链运行及实验数据输出。
agent_id	实验人员在 Trace 文件中为 Agent 指定的唯一标识（例如 agent-alice），用于区分不同 Agent 并关联其行为记录。
seed	在 agentEmuConfig.yaml 中设置的身份派生种子，系统将其与 agent_id 结合生成 DID。相同的种子与 Agent 标识会生成相同的 DID。
DID	去中心化 ID（Decentralized Identifier），用于标识 Agent 的身份，由系统根据 seed 和 agent_id 自动生成，无需在 Trace 文件中手动填写。

1.7 AgentEmulator 的系统架构

下图展示了 AgentEmulator 的模块架构。



2. 开始使用 AgentEmulator 做实验

2.1 环境准备

AgentEmulator 支持在 macOS、Linux 和 Windows 操作系统上运行，环境依赖如下表所示。

依赖	版本要求	验证命令	用途
Go	≥ 1.25	<code>go version</code>	编译运行仿真器
Python 3	≥ 3.8	<code>python3 --version</code> (Windows: <code>python --version</code> 或 <code>py -3 --version</code>)	实验后自动绘图
matplotlib / numpy / pandas	—	<code>python3 -c "import matplotlib, numpy, pandas"</code>	绘图库

代码块

```
1 # Python 绘图库缺失时安装 (Windows 系统直接使用 pip 命令)
2 pip3 install matplotlib numpy pandas
```

代码块

```
1 git clone https://github.com/HuangLab-SYSU/agent-emulator.git
2 cd agent-emulator
3 go build ./... # 首次编译, 验证环境正常
```

2.2 五分钟上手：使用 AgentEmulator 的操作流程

在 Windows 系统中，实验人员可以通过 cmd 执行 `.bat` 批处理脚本，也可以在文件资源管理器中双击运行 `run_agentemu.bat`。具体命令如下：

运行示例脚本

```
1 run_agentemu.bat
```

使用自定义配置运行示例脚本

```
1 run_agentemu.bat my-config.yaml
```

`run_agentemu.bat` 与 `.sh` 版本所做的行为完全一致：编译 → 清理旧结果 → 运行实验 → 自动绘图 → 在默认浏览器打开 HTML 图册。若 `python` 命令不可用，脚本会自动改用 `py -3`。

实验人员可随时按下 `Ctrl-C` 安全终止实验，随后 BlockEmulator-X 启动的共识节点子进程将一并被清理。

注意：实验失败时不会执行绘图步骤；另外，每次运行前，脚本文件会先清空 `figs/figs_results/` 路径中的旧实验图，绘制的实验图永远只反映最近一次成功的实验结果。

使用 **MacOS** 和 **Linux** 系统的实验人员可使用 `bash` 运行脚本，示例如下：

运行示例脚本

```
1 bash run_agentemu.sh
```

实验人员运行以上命令后，脚本会依次自动完成以下操作：

1. 编译： `go build ./...`
2. 清理旧的实验数据：删除 `./exp` 目录

3. **实验运行步骤**：读取 `agentEmuConfig.yaml` → 编译 trace 为交易数据集 → 自动启动 BlockEmulator-X 区块链（默认 4 分片，每分片 4 节点）→ 交易数据全部上链 → 实验完成，区块链自动停止
4. **自动绘制实验图**：读取最新一轮的数据记录表，生成 PNG 格式的数据图到 `figs/figs_results/` 目录中
5. **将 PNG 数据图嵌入 HTML 页面中并自动在浏览器中打开**

实验人员也可使用以下命令自定义配置运行：

使用自定义配置运行示例脚本

```
1 bash run_agentemu.sh my-config.yaml
```

2.3 编写 Trace 文件

Trace 文件采用 JSONL 格式记录 Agent 的行为序列，作为 AgentEmulator 的实验输入数据，用于模拟 Agent 行为和区块链常规交易。文件中的每条记录通过 `agent_id` 标识执行行为的 Agent，通过 `ts` 确定行为的执行顺序，并指定具体的行为类型，包括 `join`、`pay` 和 `leave`。

AgentEmulator 基于该文件，以确定性方式生成仿真实验的全部后续产物：

代码块

```
1 trace 文件（实验的全部意图）
2   |   编译：意图 → 真实区块链交易（DID 派生、nonce、数据字段）
3   ▼
4 交易数据集 → 仿真回放 → Agent 账本 / 链上测量 → 绘图展示效果
```

2.3.1 注意事项

使用 Trace 文件时，实验人员应注意以下事项：

1. **行为描述与交易生成**。实验人员通过 Trace 文件描述 Agent 的行为，无需手动构造区块链交易。AgentEmulator 根据行为记录和 `seed`，以确定性方式生成 Agent 身份标识（DID）、交易 nonce 以及相关数据字段。
2. **可复现条件与执行顺序**。在 Trace 文件、`seed` 和代码版本均相同的条件下，AgentEmulator 生成一致的交易计划，具体说明见附录 FAQ Q10。Trace 文件中的 `ts` 用于确定行为的逻辑执行顺序；对于 `ts` 相同的记录，系统按照其在文件中的出现顺序执行。`ts` 不表示交易的实际上链时间，调整其数值间隔也不等同于调整交易的上链时间。

3. **Agent 生命周期与支付约束。** Trace 文件中的 `join`、`leave` 和 `pay` 记录分别描述 Agent 的“加入”、“退出”和“支付”行为。AgentEmulator 按顺序处理这些记录，并更新 `agent_registry.json` 中各 Agent 的 `active` 状态。执行 `pay` 操作时，付款方和收款方均须处于 `active` 状态。
4. **实验数据关联与版本标记。** `request_id` 用于关联支付行为、链上交易和 Agent 账本记录，建议实验人员为其设置全局唯一值。`params_hash` 用于标记行为参数的版本，系统将该字段原样写入映射文件，以支持多轮实验的数据管理。
5. **普通区块链转账交易。** Trace 文件支持包含 `sender`、`recipient` 和 `value` 字段的普通转账记录。实验人员可以将此类记录与 Agent 行为记录组合使用，从而构造包含不同交易类型的实验负载。

Trace 文件以 JSONL 格式给出，每一行表示一个 Agent 执行的某个行为或者是普通转账交易，按 `ts`（TimeStamp）排序（`ts` 相同则按照文件中的行序进行执行）。仓库自带两个示例：`traces/minimal.jsonl`（只包含 2 个 Agent 加入 / 转账 的最小实验规模示例）、`traces/agent=100_txs=10000.jsonl`（配置文件中默认使用的 trace 文件，包含 100 个 Agent，1w 笔交易），以下展示一份 trace 文件的示例。

代码块

```
1  {"agent_id":"agent-alice","action":"join","params_hash":"doc-alice-v1","ts":1}
2  {"agent_id":"agent-bob","action":"join","params_hash":"doc-bob-v1","ts":2}
3  {"agent_id":"agent-alice","action":"pay","target":"agent-
   bob","amount":12,"request_id":"payment-1","ts":3}
4  {"agent_id":"agent-bob","action":"leave","params_hash":"exit-bob","ts":5}
```

2.3.2 Trace 文件支持的行为类型

类型	含义	编译成的交易
<code>join</code>	Agent 注册“加入”	DID <code>register</code> 智能合约调用
<code>pay</code>	向另一个 Agent 转账	普通转账交易
<code>leave</code>	Agent 注销“退出”	DID <code>revoke</code> 智能合约调用
普通转账交易	由区块链账户而非 Agent 发起的转账交易	普通转账交易

普通转账交易行中不包含 `action` 字段，靠 trace 文件中的结构（`sender` 字段中使用的是账户地址）自动识别，只有三个输入字段，该普通转账交易的执行顺序严格按照 trace 文件中的行序执行，示例如下：

代码块

```
1 {"sender":"0xabc...", "recipient":"0xdef...", "value":"12345"}
```

2.3.3 Trace 文件编写时应该遵守的规则

1. `pay` 行为的双方 Agent (`agent_id` 和 `target`) 当时必须处于 `active` 状态 (已 join 且未 leave) , 否则整场仿真实验将报错终止。
2. `leave` 操作只能作用于处于 `active` 状态的 Agent; 已经 `leave` 的 Agent 再次 `join` 需要重新注册。
3. `amount` 字段的填写必须为正整数。链上新账户地址会自动获得 10^{36} wei 的初始余额, 正常实验无需担心账户余额不足。
4. **trace 文件中不需要写 DID** —— Agent 的身份由 `seed + agent_id` 确定性派生, 同 seed、同 ID 永远得到同一 DID。
5. `request_id` 建议全局唯一, 因为后续需要使用它来检索每个 Agent 的“支付意图”数据。

2.4 实验参数配置说明

AgentEmulator 包含两层参数配置: 1) Agent 侧的参数配置, 由 `agentEmuConfig.yaml` 配置文档体现; 2) 底层区块链的参数配置, 由 `config.yaml` 负责对 BlockEmulator-X 中区块链相关的参数进行配置。

2.4.1 对 Agent 侧的配置: `agentEmuConfig.yaml`

代码块

```
1 base:
2   blockemulator_config: ./config.yaml # BlockEmulator-X 实验模板: 分片数/节点数/
   共识类型等
3   result_dir: ./exp/agentemu-results # 全部 agentSupervisor 产物的根目录
4   module_root: "." # 仓库根目录 (构建区块链二进制用)
5 experiment:
6   seed: 20260903 # DID 派生种子
7   trace: ./traces/agent=100_txs=10000.jsonl # trace 文件路径
8 chain:
9   enabled: true # true: 交易数据集产出后自动启动区块链上链
10  run_timeout_seconds: 600 # 整场上链实验的超时设置, 实验人员可根据交易
   数量动态调整
```

```

11     node_exit_grace_seconds: 15                # supervisor 退出后节点的退出窗口期
12     loop:
13         max_rounds: 1                        # 轮次上限（多轮反馈为预留接口，默认单轮）
14     protocols:
15         pay:
16             plugin: direct-pay                # 目前仅支持逐笔直付
17         identity:
18             plugin: did-simple
19             contract_address: "0x0000000000000000000000000000000000000000000000000000000000000030"

```

2.4.2 对 BlockEmulator-X 侧的配置：config.yaml（默认使用 BlockEmulator-X 的原生配置）

agentSupervisor 以 `config.yaml` 文件为模板，用于为每次实验生成独立的 BlockEmulator-X 区块链配置。系统自动调整数据存储路径和日志路径，将交易源设置为 `plan_source`，并将 `tx_number` 设置为 Trace 文件对应的交易数量。实验人员可根据需要修改以下常用配置项：

- `system.shard_num` / `system.node_num`：进行实验的区块链分片数量，以及每个分片中的节点数量
- `consensus_node.block_interval`：出块间隔（ms）
- `system.log.log_level`：`debug` / `info` / `warn` / `error` 4种日志等级

```

system:
### -----
### `shard_num` is the number of shards. It should be a positive integer.
shard_num: 4
### -----

### -----
### `node_num` is the number of nodes per shard. It should be a positive integer.
node_num: 4
### -----

### -----
### `limit` is the limitation of bytes/transactions of a block.
limit: 5000
### -----

### -----
### `consensus_type` is the type of consensus. Optional setting: [static_relay, static_broker, clpa
### `static_relay`: The location distribution of accounts is static, and accounts will not be migrat
### `static_broker`: The location distribution of accounts is static, and accounts will not be migrat
### `clpa_relay`: The locations distribution of accounts is dynamic. When each epoch ends, some acco
### `clpa_broker`: The locations distribution of accounts is dynamic. When each epoch ends, some acc
consensus_type: "static_relay" # [static_relay, static_broker, clpa_relay, clpa_broker]
### -----
💡
log:
### -----
### `log_dir` is the root directory for the logger. It should be a path.
### The actual log directory of a node (ShardID=2, NodeID=1) is "`log_dir`/logs/shard=2_node=1".
log_dir: "./exp/"
### -----

### -----
### `log_level` is level for the logger. Optional setting: [debug, info, warn, error]
### `debug` is usually used when debugging, and it shows all levels of logs.
### `info` is the most commonly used level. It shows [info, warn, error] levels of logs.
### `warn` shows [warn, error] levels of logs.
### `error` shows only the [error] level of logs.
log_level: "info" # [debug, info, warn, error]
### -----

```

BlockEmulator-X 侧配置文件示例

2.5 启动运行实验与观察实验结果相关的配置

实验人员可通过以下命令启动实验，具体详情见 2.2 节。

代码块

```
1 bash run_agentemu.sh # 或 bash run_agentemu.sh <配置文件>
```

- **BlockEmulator-X** 区块链运行过程中的日志会**实时镜像到控制台**中输出。同时，完整的实验日志会保存在 `exp/agentemu-results/round_001/chain/logs/` 目录中。
- 如果实验人员只想预览编译出的交易数据集，而不启动 **BlockEmulator-X** 运行区块链进行实验，请在 `agentEmuConfig.yaml` 配置文件中把 `chain.enabled` 改为 `false` 即可（该配置下，`agentSupervisor` 仅将 `trace` 文件编译成交易数据集，而不会启动 **BlockEmulator-X** 运行区块链）
- 实验结束时，实验运行脚本文件会在控制台中打印实验运行结果的输出路径，与使用 `python` 脚本绘制的实验图的存储路径

2.6 实验结束后输出的文件一览

2.6.1 实验结束后生成的 results 文件

代码块

```

1  exp/agentemu-results/
2  |— agent_registry.json          # agent_id ↔ DID 映射与 active 状态
3  |— rounds_summary.json         # 每轮记录数/交易数汇总
4  |— round_001/
5      |— agent_transactions.jsonl # 交易数据集 (hash/双方/金额/nonce/data)
6      |— agent_action_txs.jsonl  # ★ 动作→交易映射 (意图 ↔ 链上 hash)
7      |— Agent_Events.csv        # Agent 行为事件流
8      |— agents/                 # ★ 每 Agent 账本 (绘图的输入)
9          |— agent-001.csv
10         |— ...                 # 每个 active Agent 对应一个文件
11     |— chain/                  # 本次实验区块链运行相关结果
12         |— config.yaml / ip_table.json
13         |— logs/               # BlockEmulator-X 启动的各节点运行时产生的日志
14         |— results/
15             |— relay_stats_detail_tx_info.csv # ★ 链上每笔交易的生命周期
16             |— relay_stats_brief_info.csv    # 按 epoch 的 TPS/TCL 汇总

```

2.6.2 数据记录表 CSV (`agents/agent-XXX.csv`) 格式说明

该数据记录表中的一行对应 Agent 的一次操作，数据格式如下：

数据记录表表头

```
1  block_height, tx_hash, sender, recipient, value, balance, block_time_ms
```

- `balance` 为该交易执行后对应 Agent 的余额，数值约 10^{36} (超出 `int64` 数据类型可表示的范围，需按字符串读入)
- `block_time_ms` 为打包该交易的区块出块时间
- 数据记录表中的行序按照区块出块时间排序，同一时间内以“分片/高度/块内”的序号决定最后的顺序

	block...	tx_hash	sender	recipient	value	balance	block_time_ms
1	4	091375efe6cb7ad5c...	e2ff98eb11e21278...	0000000000000000...	0	100000000000000000...	1789791424608
2	4	8dacbc97013434b74...	493e8aa29e00f101...	e2ff98eb11e21278...	896	100000000000000000...	1789791424608
3	4	e1ee65b2c51a887a8...	e2ff98eb11e21278...	8ff496b82d49b429...	557	100000000000000000...	1789791424608
4	4	7c2af51d7879ea2e2...	e2ff98eb11e21278...	550f57b52fdbcb30b...	7	100000000000000000...	1789791424608
5	4	9e82140f4dbc13bc7...	e2ff98eb11e21278...	ab2e08f5af6c9ab4...	263	100000000000000000...	1789791424608
6	4	e131c5681d2cbf5b2...	e2ff98eb11e21278...	3f408e7398f394a4...	713	999999999999999999...	1789791424608
7	4	5a2622bc04bf2fa04...	e2ff98eb11e21278...	4d0645a87a11f78c...	719	999999999999999999...	1789791424608
8	4	4c1d1b191037ff403...	e2ff98eb11e21278...	cab5ddffa9cd588f...	133	999999999999999999...	1789791424608
9	4	5dbea2c42d7c1fd0d...	e2ff98eb11e21278...	78c95ffc2f894603...	136	999999999999999999...	1789791424608
10	4	da5a806a6a8770d27...	3f408e7398f394a4...	e2ff98eb11e21278...	447	999999999999999999...	1789791424608
11	4	ff65c4ad663689265...	e2ff98eb11e21278...	b97d39164fbc65b3...	957	999999999999999999...	1789791424608
12	4	7cbb1e0718767e8ef...	e2ff98eb11e21278...	4d0645a87a11f78c...	185	999999999999999999...	1789791424608
13	4	c981082a5740a548d...	36c28b8468d76ab9...	e2ff98eb11e21278...	128	999999999999999999...	1789791424608
14	4	f01e15d172e6bec55...	e2ff98eb11e21278...	8ff496b82d49b429...	296	999999999999999999...	1789791424608
15	4	fb858f3f19991916...	e2ff98eb11e21278...	5834db70f7adbd86...	53	999999999999999999...	1789791424608

Agent 数据记录表

`agent_action_txs.jsonl` 文件的示例如下（每个 Agent 动作用一行来表示）：

代码块

```
1 {"seq":3,"action":"pay","agent_id":"agent-alice","target":"agent-bob",
  "amount":12,"ts":3,"request_id":"payment-1","params_hash":"","tx_hashes":
  ["f58c994e..."]}
```

1	{	"seq":1,"action":"join","agent_id":"agent-001","target":"","amount":0,"ts":1,"request_id":"","params_hash":"doc-agent-001-v1","tx_hashes":["091375efe6cb7ad5c4edc766b767ad904a38f1e0e21 Analyzing...
2	{	"seq":2,"action":"join","agent_id":"agent-002","target":"","amount":0,"ts":2,"request_id":"","params_hash":"doc-agent-002-v1","tx_hashes":["72969e174301277ecd5b76859bf797463565a47ddc6e236e51a1
3	{	"seq":3,"action":"join","agent_id":"agent-003","target":"","amount":0,"ts":3,"request_id":"","params_hash":"doc-agent-003-v1","tx_hashes":["e733b68e5a1a516bcb815e1e3033220a6e3f136f1291eb7b61b6
4	{	"seq":4,"action":"join","agent_id":"agent-004","target":"","amount":0,"ts":4,"request_id":"","params_hash":"doc-agent-004-v1","tx_hashes":["4c21c04ca8f171b46b449ac5be8b20e3bb03da5dbfcd8419f233
5	{	"seq":5,"action":"join","agent_id":"agent-005","target":"","amount":0,"ts":5,"request_id":"","params_hash":"doc-agent-005-v1","tx_hashes":["7770d583aa33caa1c0214cbb2665186abc23a2e3884a17537be
6	{	"seq":6,"action":"join","agent_id":"agent-006","target":"","amount":0,"ts":6,"request_id":"","params_hash":"doc-agent-006-v1","tx_hashes":["0bbd8f39ae033f6ba73b045065ef590be1f6e15fd963756c9a01
7	{	"seq":7,"action":"join","agent_id":"agent-007","target":"","amount":0,"ts":7,"request_id":"","params_hash":"doc-agent-007-v1","tx_hashes":["2dc0ca24eb5ddee4abc47c817722dbe5242b61503f18ce7408ea7
8	{	"seq":8,"action":"join","agent_id":"agent-008","target":"","amount":0,"ts":8,"request_id":"","params_hash":"doc-agent-008-v1","tx_hashes":["b076c1f220db0e3f7ffea8800cccd483d2d68afcf5ee52cb9264
9	{	"seq":9,"action":"join","agent_id":"agent-009","target":"","amount":0,"ts":9,"request_id":"","params_hash":"doc-agent-009-v1","tx_hashes":["0673b3f4e34e4af1a6eaeacaf5c8a854ce0f1ab2f6ed7c08d3
10	{	"seq":10,"action":"join","agent_id":"agent-010","target":"","amount":0,"ts":10,"request_id":"","params_hash":"doc-agent-010-v1","tx_hashes":["2a36e3fe0273f8cd9a4628ec9c61d8687abdcc04af3cc785de
11	{	"seq":11,"action":"join","agent_id":"agent-011","target":"","amount":0,"ts":11,"request_id":"","params_hash":"doc-agent-011-v1","tx_hashes":["e0dc3535c74bde7a74e2a159164a48e434bf5442f8e4fc9f6e
12	{	"seq":12,"action":"join","agent_id":"agent-012","target":"","amount":0,"ts":12,"request_id":"","params_hash":"doc-agent-012-v1","tx_hashes":["9a41067cc9b0e801f71f1a074a1f4a25dc34b6b18d0b978873
13	{	"seq":13,"action":"join","agent_id":"agent-013","target":"","amount":0,"ts":13,"request_id":"","params_hash":"doc-agent-013-v1","tx_hashes":["ed122804c105abd562850f17d70ef3fc674065678969b548e5
14	{	"seq":14,"action":"join","agent_id":"agent-014","target":"","amount":0,"ts":14,"request_id":"","params_hash":"doc-agent-014-v1","tx_hashes":["cd2a7e5debc99b51646834e7dac24679bdac9773973865f9e

Agent 操作记录文件示例

2.7 实验结果的展示

每次实验成功完成后，AgentEmulator 自动读取本轮实验的 Agent 账本 CSV 文件，生成论文排版风格的余额变化图，并以 PNG 格式保存至 `figs/figs_results/` 目录。随后，系统将图表整合为支持中英文切换的静态 HTML 图册页面（`figs/figs_results/index.html`），并在默认浏览器中自动打开该页面。

以下各小节依次介绍自动绘图流程（2.7.1）、四张实验图表的展示内容（2.7.2）、图册页面的使用方法（2.7.3），以及无需重新运行实验的手动绘图方法（2.7.4）。

2.7.1 实验结束后，模拟器自动执行如下操作

`run_agentemu.sh`（Windows 下为 `run_agentemu.bat`）脚本文件在实验成功结束后将自动执行以下操作：

1. 定位 `exp/agentemu-results/` 路径下 round 编号最大一轮的 `agents/` 目录
2. 清空 `figs/figs_results/` 里的旧实验图与旧 `index.html` 文件
3. 运行 `figs/python_code/plot_agent_balance.py` 生成全部实验图的 PNG 格式图片
4. 运行 `figs/python_code/build_fig_html.py` 生成展示实验结果图表的 html 页面
5. 模拟器调用网页打开命令在实验人员计算机的默认浏览器中打开 `figs/figs_results/index.html` 页面，展示绘图结果。根据实验人员设备的操作系统类型不同，网页打开命令具体如下：macOS 使用 `open` 命令，Linux 使用 `xdg-open` 命令，Windows 使用 `start` 命令。

2.7.2 实验图绘制的内容

实验结果图表的 HTML 页面中实验图编号	内容	对应的 PNG 文件
图 1	全部 Agent 余额变化总览：左图“按 Agent ID”，右图“按 Δ balance 升序”展示所有 Agent 余额变化的柱状图	<code>fig1_all_agents_overview.png</code>
图 2	按照交易顺序统计所有 Agent 的余额分布。主要过程如下：先按照 tx_hash 去重后按确定性顺序回放每个 Agent 的余额变化，然后展示所有 Agent 余额变化最小/最大位置线、四分位线与均值线	<code>fig2_global_tx_order.png</code>

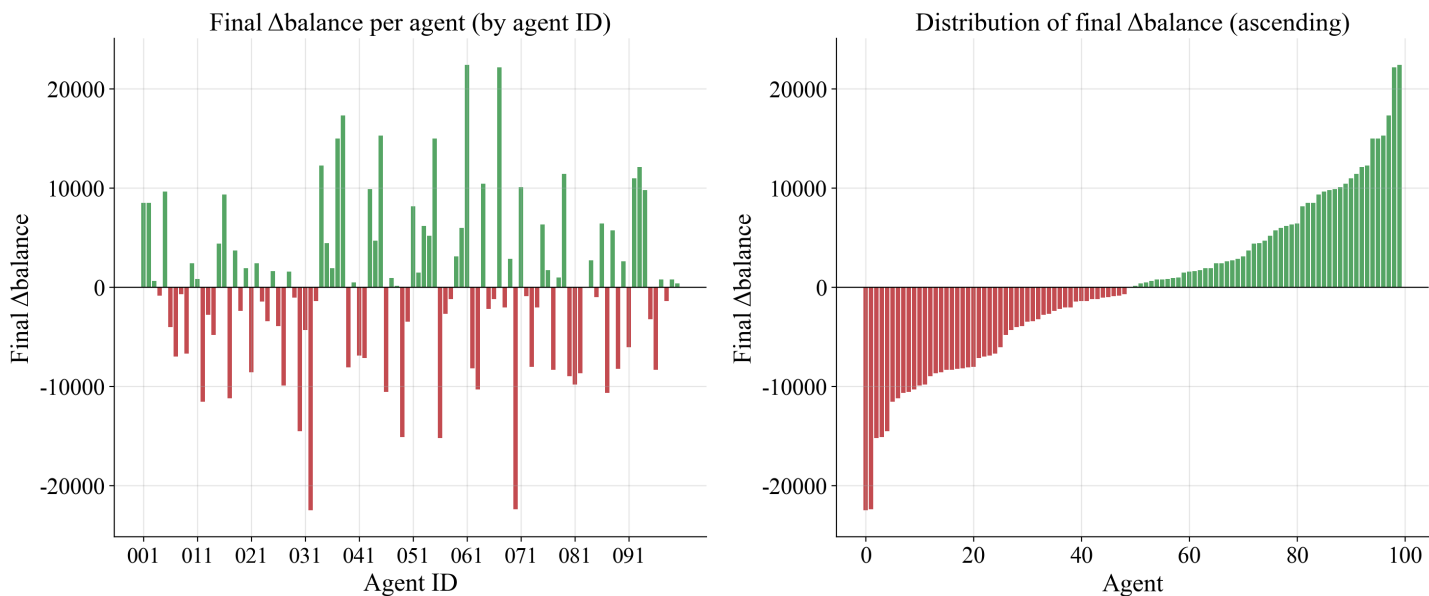
图 3	按不同 Agent 的交易进度归一化对齐：各 Agent 自身交易序号拉伸到 0-1 后叠加，另附所有 Agent 的交易处理完成后所有 Agent 余额变化均值标注	fig3_normalized_progress.png
图 4	分组展示各个 Agent 的余额变化：每 5 个 Agents 一张子图，使用虚线标注每个 Agent 所在分片相邻两个区块的分界点	fig4_agents_001-005.png ... fig4_agents_096-100.png

请注意，所有实验图绘制的是**相对初始余额的变化量 $\Delta\text{balance} = \text{balance} - \text{初始余额}$** 。

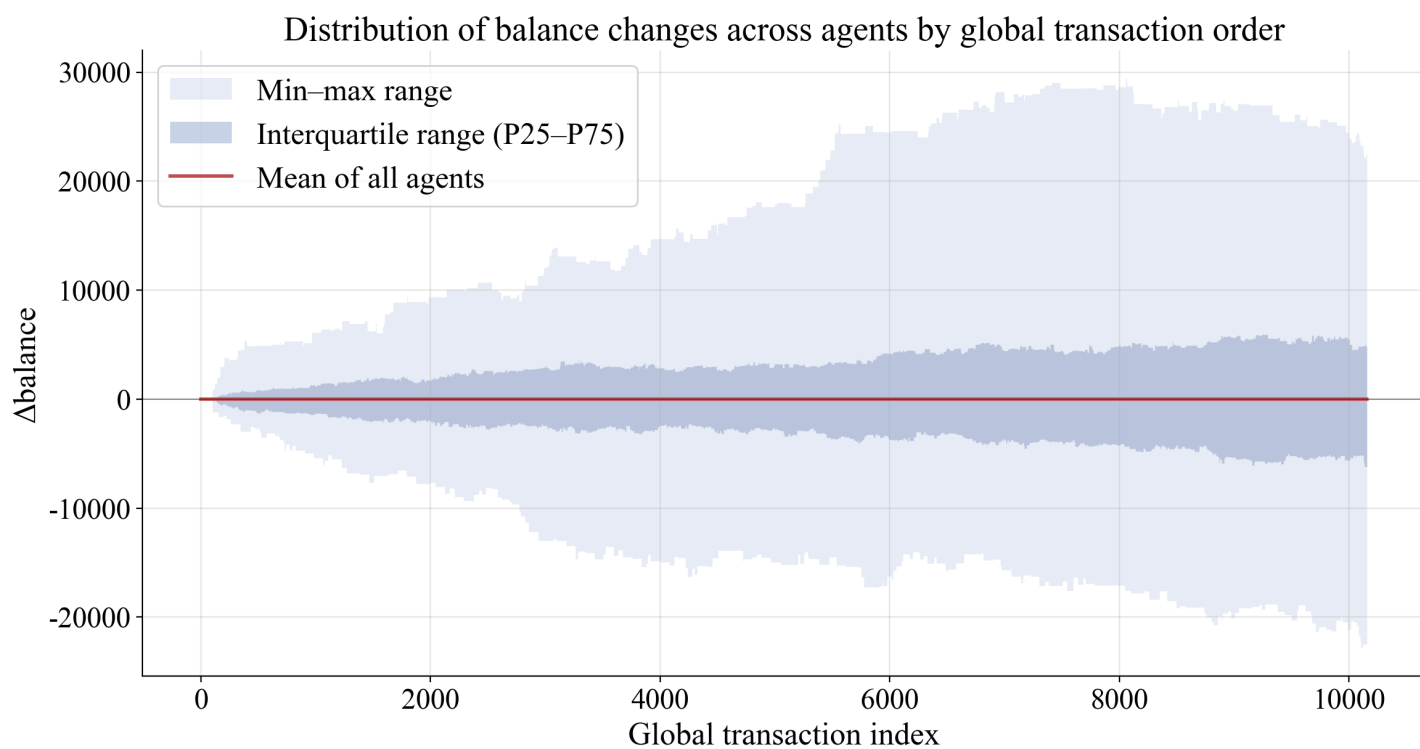
以下展示的实验结果是在一台 Mac mini (macOS 15.6) 上运行的。该 Mac mini 的硬件为 Apple M4 Pro 芯片 (12 核：8 个性能核 + 4 个能效核)、24 GB 内存。区块链环境基于黄华威研究组自研的 BlockEmulator-X (Go 语言版本为 1.25.7, GitHub 代码仓库地址为 github.com/HuangLab-SYSU/block-emulator-x)。

本次实验采用单机多进程模拟部署，实验相关设置如下：

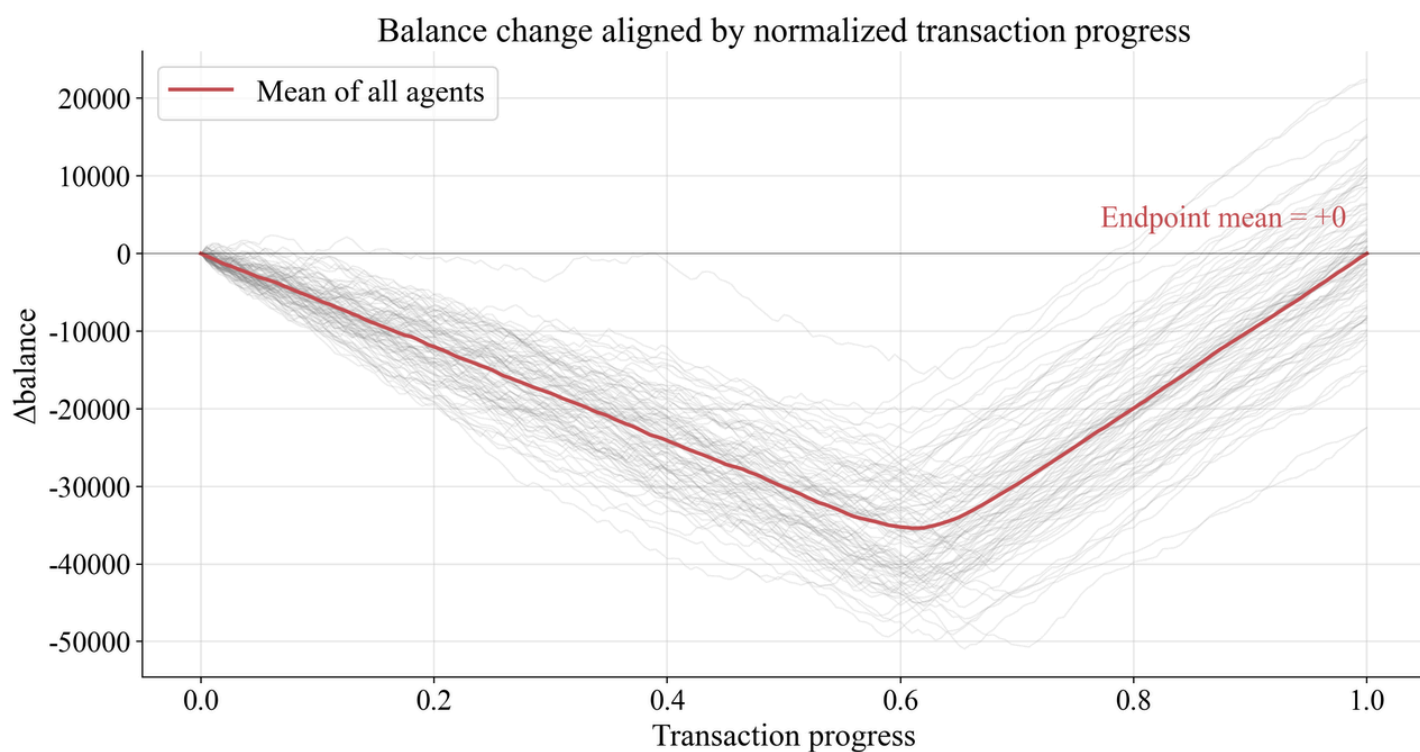
- 1) 本次实验共模拟 4 个分片、每分片 4 个共识节点 (合计 16 个共识节点) 外加 1 个 supervisor 节点，均以 127.0.0.1 IP 下的不同端口 (32217-32547, BlockEmulator-X 中的 supervisor 节点端口号为 38800) 通过 direct 模式通信；
- 2) 共识与跨分片协议为 `static_relay` (账户分布静态、跨片交易由 Relay 处理)，出块间隔设置为 2000 ms，交易池按交易数量打包，每块上限 5000 笔；
- 3) 区块存储采用 BoltDB，世界状态采用以太坊式 LevelDB，布隆过滤器位图长度 4096；
- 4) 在 AgentEmulator 实验层以随机种子 20260903 回放包含 100 个 Agent、10,000 笔交易的 trace (`plan_source` 方式)，每轮实验自动编译并重新启动 BlockEmulator-X 区块链模拟实验。



Agents 余额变化图

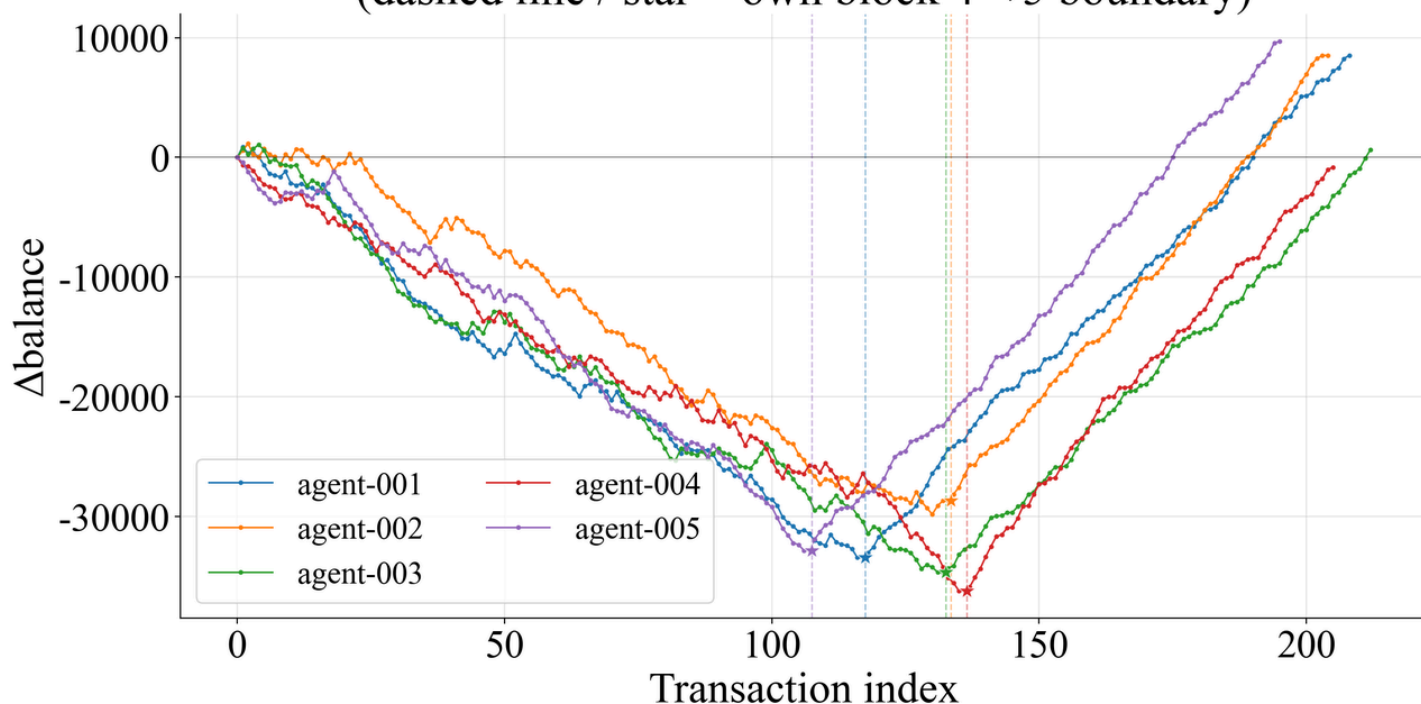


Agents 余额变化分布图



所有 Agents 余额变化总览图

Agents 001–005: balance change
(dashed line / star = own block 4→5 boundary)

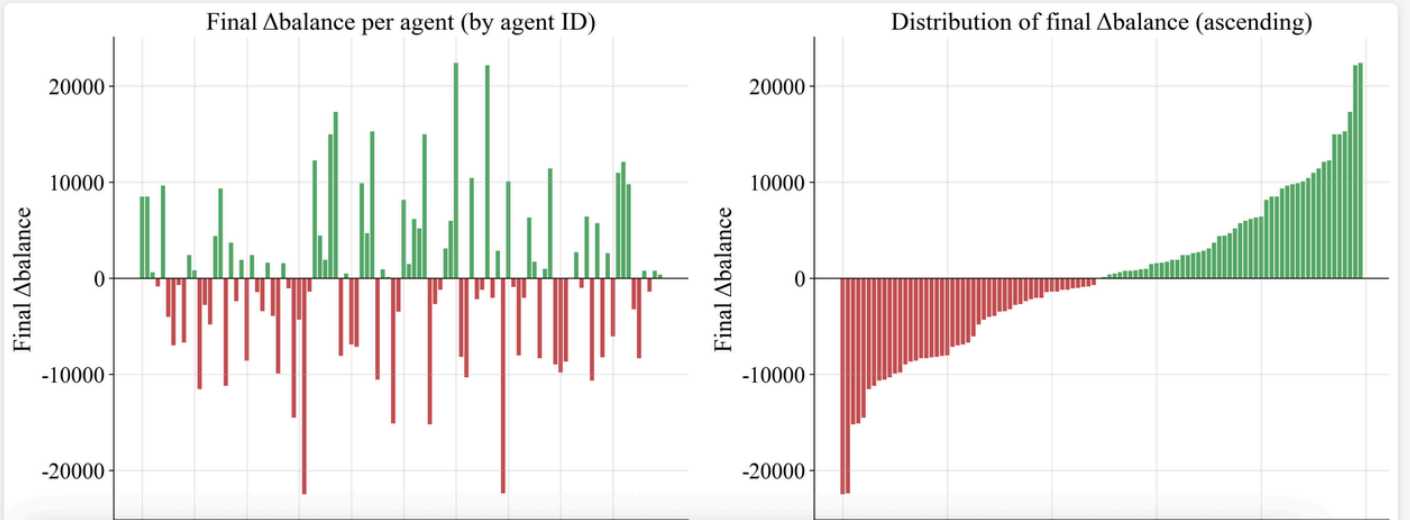


不同 Agent 余额变化分组展示图

2.7.3 用于展示实验的 HTML 页面的 UI 设计

- 顶部深色页眉：标题、生成时间、图表数量、数据来源目录与 Agent 数
- 绘制出的实验图中，图 1/2/3 整幅展示，图 4 为所有 Agents 按照每 5 个 Agent 为一组绘制的 Agent 余额变化图；实验人员可[点击任意图片即可打开对应的原图](#)
- **中英文切换**：右上角按钮（当前中文时显示 "EN"，英文时显示 "中文"），或按键盘 **L** 键；切换作用于页面标题、章节标题、元信息与页脚，语言偏好自动记忆，下次打开该 html 页面时保持上一次的设置
- 实验图内文字由绘图脚本生成，不随页面语言切换变化

Fig. 1 · Overview of All Agents



HTML 页面 UI 设计总览图

2.7.4 手动 / 独立运行绘图脚本

实验人员不重跑整个实验也可以随时使用以下命令重新画图。绘图脚本路径按

`figs/python_code/` 相对路径进行定位，在 AgentEmulator 项目根目录下运行以下命令均可：

代码块

```
1  # 默认：自动选取最新一轮实验结果，输出到 figs/figs_results/
2  python3 figs/python_code/plot_agent_balance.py
3  python3 figs/python_code/build_fig_html.py
4
5  # 指定数据目录与输出目录（例如用历史快照重新绘制）
6  python3 figs/python_code/plot_agent_balance.py \
7      --data-dir exp/agentemu-results/round_001/agents \
8      --fig-dir figs/figs_results
9  python3 figs/python_code/build_fig_html.py \
10     --data-dir exp/agentemu-results/round_001/agents
11
12 # 只生成页面不重新画图
13 python3 figs/python_code/build_fig_html.py
14
15 # 打开图册
16 open figs/figs_results/index.html # macOS
```

Windows 下等价命令（实验人员可通过 cmd 运行以下命令，路径使用反斜杠；`python` 不可用时改用 `py -3`）：

代码块

```
1 python figs\python_code\plot_agent_balance.py
2 python figs\python_code\build_fig_html.py
3
4 python figs\python_code\plot_agent_balance.py --data-dir exp\agentemu-
  results\round_001\agents --fig-dir figs\figs_results
5 python figs\python_code\build_fig_html.py --data-dir exp\agentemu-
  results\round_001\agents
6
7 rem 打开图册
8 start "" figs\figs_results\index.html
```

两个绘图脚本的参数说明如下表：

脚本	参数	默认值	说明
plot_agent_balance.py	--data-dir	自动选最新一轮 agents/	Agent CSV 所在目录
	--fig-dir	figs/figs_results/	绘图结果的 PNG 图片输出目录
build_fig_html.py	--fig-dir	figs/figs_results/	扫描 PNG 并在此生成 index.html 页面
	--data-dir	无	仅用于页面显示数据来源与 Agent 数量

2.8 对实验结果进行分析

实验人员可通过实验产生的部分结果进行联合解析，即可回答“某笔支付是否确认、何时确认”的问题。解析路径为：agent_action_txs.jsonl（request_id → tx hashes）→ relay_stats_detail_tx_info.csv（hash → 提交时间）。具体的代码如下：

代码块

```
1 import json, csv
2
3 chain = {}
4 with open('exp/agentemu-
  results/round_001/chain/results/relay_stats_detail_tx_info.csv') as f:
5     for row in csv.DictReader(f):
6         chain[row['OriginalHash']] = row
```

```

7
8 total = confirmed = 0
9 for line in open('exp/agentemu-results/round_001/agent_action_txs.jsonl'):
10     a = json.loads(line)
11     if a['action'] != 'pay':
12         continue
13     total += 1
14     if all(h in chain for h in a['tx_hashes']):
15         confirmed += 1
16         # 例：查某笔的确认时间
17         # print(a['request_id'], chain[a['tx_hashes']][0]['Tx finally commit
time'])
18
19 print(f'支付意图 {confirmed}/{total} 全部确认')

```

每笔交易的延迟统计口径为：`Tx finally commit time - Tx create time`；对于跨片交易，在实验结束生成的 CSV 结果表格中使用了 Relay1/Relay2 两段 提议/提交 时间对跨片交易的处理进行细分。

实验人员也可以直接使用 pandas 库对 Agent 账本进行分析，相关代码如下：

代码块

```

1 import pandas as pd
2 df = pd.read_csv('exp/agentemu-results/round_001/agents/agent-001.csv',
3                 dtype={'balance': str, 'value': str})

```

3. 常见问题（FAQ）

Q1：实验人员重新启动 AgentEmulator 运行实验时，报 `file already exists: .../block_record.csv` 错误？

出现这个问题表示上一次使用 AgentEmulator 进行实验的 results 文件没有清理，而这些 results 文件都是独占创建的。实验人员可使用 `run_agentemu.sh` 脚本文件启动实验则不会出现该问题。如果实验人员想通过手动运行 `go run cmd/agentemu/main.go` 命令启动 AgentEmulator，则需要先执行 `rm -rf exp/agentemu-results` 命令清理旧实验产生的 results 文件。

Q2：为什么有时 join 操作没有生成注册交易？

join 操作只有在 Agent **首次加入或离开后重新加入**时才会生成对应的注册交易。出现这个问题，实验人员可在 `agent_registry.json` 文件中查找到该 Agent 是否已是 active 状态。如果实验人员想复现所有 agents 在 join 时生成对应的注册交易，则需要将旧的 results 文件删除。

Q3：Agent 之间的互相转账是通过调用智能合约实现的吗？

不是。Agent 之间的 `pay` 操作会被 agentSupervisor 编译为 **普通转账交易**；只有 Agent 的 `join / leave` 操作是通过调用智能合约实现的。不过，相关的智能合约当前暂未部署，调用相关的智能合约目前只是在 EVM 层做了一个记录，而非真实调用智能合约的状态。但 Agent 之间的 `pay` 操作的转账操作是真实有效的。

Q4：控制台日志输出太多，无法查看到关键信息怎么办？

实验人员可在 `config.yaml` 配置文件中，修改 `system.log.log_level` 部分的配置为 `warn`，或在运行脚本文件时使用 `bash run_agentemu.sh > run.log 2>&1` 命令进行重定向，即可减少控制台输出的日志内容。

Q5：实验人员如何运行单分片的小规模实验？

实验人员可将 `config.yaml` 中的 `system.shard_num` 设置为 `1`，使区块链以单分片模式运行，并根据实验需求调整 Trace 文件中的行为记录和交易数量，以控制实验的输入规模。

Q6：实验运行结束但是没有弹出 HTML 页面是什么原因？

实验人员可先查看控制台末尾是否有输出 `figures & gallery:`
`./figs/figs_results/index.html`：

- 没有输出这行且出现 `warn: no agent CSVs ...`：表示本次实验没有生成 Agent 相关的数据记录（例如实验人员在 `agentEmuConfig.yaml` 文件中将配置设置为 `chain.enabled: false`，表示只通过 agentSupervisor 将 trace 文件编译成对应的交易数据，而不启动 BlockEmulator-X 模拟区块链运行），属于正常现象；
- 有这行但 HTML 页面没有弹出：实验人员可手动使用以下命令 `open figs/figs_results/index.html` 打开该 HTML 页面。

Q7：绘图脚本提示 `ModuleNotFoundError: matplotlib`，应如何处理？

该错误表示当前运行绘图脚本的 Python 环境缺少 `matplotlib` 库。实验人员应在该环境中执行 `pip3 install matplotlib numpy pandas` 命令，安装绘图所需的依赖库；也可以使用已安装上述依赖库的 Python 解释器运行脚本。

Q8：实验人员如何使用历史数据重新绘制实验图？

实验人员可将 `--data-dir` 参数设置为目标实验轮次的 `round_XXX/agents/` 目录，并手动执行绘图脚本。由于 `run_agentemu.sh` 每次运行时都会清空 `exp/` 和 `figs/figs_results/` 目录，实验人员应在运行前备份需要保留的历史数据和图表。

Q9：实验图图 4 包含较多子图，实验人员如何调整 Agent 的展示范围或分组方式？

当前绘图脚本按照每组 5 个 Agent 的规则生成实验图图 4 的子图。实验人员可修改 `figs/python_code/plot_agent_balance.py` 文件中图 4 对应的绘图代码，筛选需要展示的 Agent，或调整分组大小。其中，`range(0, len(dfs), 5)` 中的 5 表示每组包含的 Agent 数量。

Q10：AgentEmulator 的实验结果是否具有可复现性？

在 `seed`、Trace 文件和代码版本均相同的条件下，AgentEmulator 生成的交易数据集文件和 Agent 行为 CSV 文件逐字节一致，Agent 数据记录也按照确定性的全局顺序生成。链上交易的打包时序受运行时条件影响，这一特性与 BlockEmulator-X 一致。

Q11：Windows 系统提示“python 不是内部或外部命令”，或执行 `python` 后打开 Microsoft Store，应如何处理？

上述情况通常表示系统尚未安装可用的 Python，或 Python 的安装路径未加入 `PATH` 环境变量。实验人员可从 [Python 官方网站](#) 下载安装程序，并在安装时勾选“Add python.exe to PATH”。

如果系统已安装 Python 启动器，实验人员也可以使用 `py -3` 调用 Python 3；

`run_agentemu.bat` 脚本中已提供自动回退至该启动器的逻辑。安装完成后，实验人员应使用所选解释器安装绘图依赖库：使用 `python` 时，执行 `python -m pip install matplotlib numpy pandas`；使用 `py -3` 时，执行 `py -3 -m pip install matplotlib numpy pandas`。

研究团队

本文由中山大学软件工程学院黄华威教授研究组 (HuangLab) 主导完成。研究组长期专注于区块链分片、共识协议、链上金融、AI 与区块链交叉等方向的研究，已在 IEEE/ACM ToN、TSC、TC、TPDS、TDSC、INFOCOM、WWW、ICDCS、SRDS 等期刊与会议发表多篇区块链论文。

AI 智能体可信基础设施的仿真与实验平台 AgentEmulator 的 GitHub 仓库：

<https://github.com/HuangLab-SYSU/agent-emulator>

开源区块链分片实验平台 BlockEmulator 主页：

<https://www.blockemulator.com>

BlockEmulator 的 GitHub 仓库：

<https://github.com/HuangLab-SYSU/block-emulator>

HuangLab 近 7 年专注于区块链分片（Blockchain Sharding）理论与技术架构。若您对区块链分片、共识协议、DeFi 协议等方向感兴趣，欢迎关注

HuangLab 公众号（ID: Huang-Lab），

或访问 HuangLab 学术主页：

<http://xintelligence.pro>